

AI-Patch-Gate für AAOS unter Windows

Benutzerhandbuch

1. Überblick

Wozu das Werkzeug dient

Ein AAOS-Build dauert lange. Ein Tippfehler im Diff, ein vergessener Null-Check oder eine Änderung, die an der eigentlichen Absicht vorbeigeht, fällt oft erst nach vielen Minuten auf, wenn der Compiler abbricht oder das Image nicht tut, was es soll. Der AI-Patch-Gate schaltet eine kurze Vorprüfung vor den Build: Ein KI-Modell sieht sich die aktuellen Änderungen einmal pro m an und sagt, ob es einen Compile-Bruch, einen konkreten Laufzeitfehler oder eine verfehlte Absicht erwartet.

Der Compiler bleibt die eigentliche Prüfung. Das Modell kennt keine absolute Wahrheit, und der Gate hält Sie nie dauerhaft auf. Ein Urteil, eine abgelehnte Frage und ein früher fehlgeschlagener Stand sperren den nächsten Build nicht.

Was der Gate tut

- 1 Sie rufen wie gewohnt m in Ihrer WSL-Distribution auf.
- 2 Der Gate sammelt den Diff gegen HEAD, den obersten Stash und die Betreffzeilen Ihrer letzten Commits. Nur Dateien, die er noch nicht gesehen hat, gehen an das Modell.
- 3 Das Modell antwortet mit drei Einschätzungen: Compile (ok, likely_fail, unknown), Laufzeit (ok, concern, unknown) und Effekt (likely, unlikely, unknown). Wenn ein Mensch bestätigen sollte, stellt es eine Rückfrage in einem Satz.
- 4 Je nach Betriebsart fragt der Gate vor dem Build nach, oder er startet den Build sofort und zeigt den Hinweis nebenher an, unter Windows in einem eigenen Fenster.
- 5 Nach dem Build merkt sich der Gate, ob dieser Stand kompiliert hat. Ein Stand, der schon einmal durchlief, wird nicht noch einmal geschickt.

Was Sie brauchen

- Windows mit einer WSL-Distribution, in der Ihr AAOS/AOSP-Quellbaum liegt. Der Baum enthält build/soong/bin/m, build/soong/soong_ui.bash und build/envsetup.sh.
- In der Distribution das Paket jq. Für den Weg über ein API-Token oder eine eigene URL zusätzlich curl. Beispiel: `sudo apt install jq curl`.
- Einen KI-Anbieter: Claude, Microsoft Copilot oder Antigravity. Entweder ist dessen Kommandozeile (c laude, copilot, agy) in der Distribution installiert und angemeldet, oder Sie haben ein API-Token. Microsoft Copilot braucht die Kommandozeile copilot auch mit Token.
- Windows PowerShell 5.1 für den grafischen Installer. Er liegt im Verzeichnis aaos-ai-gate neben diesem Handbuch.

In fünf Minuten einsatzbereit

- 1 Öffnen Sie PowerShell im Verzeichnis aaos-ai-gate und starten Sie den Installer:

```
powershell -NoProfile -STA -ExecutionPolicy Bypass -File .\install-windows.ps1
```

- 2 Tragen Sie den Quellbaum ein, am besten als \\wsl\$\\<Distribution>\\...-Pfad, zum Beispiel \\wsl\$\\Ubuntu\\home\\<name>\\aosp.

- 3 Lassen Sie Betriebsart, Anbieter und Anmeldung auf den Vorschlägen. Der Installer hat in der Distribution schon nachgesehen, welche Kommandozeile installiert ist.
- 4 Klicken Sie auf Installieren. Das Ergebnisfeld zeigt, was in den Baum geschrieben wurde.
- 5 Öffnen Sie ein neues WSL-Terminal und bauen Sie wie immer:

```
source build/envsetup.sh
lunch sdk_car_x86_64-userdebug
m
```

Beim ersten `m` mit Änderungen meldet sich der Gate mit Zeilen, die mit `ai-patch-gate:` beginnen. In der blockierenden Betriebsart erscheint die Frage `So beabsichtigt? [j/N]`, wenn das Modell etwas auszusetzen hat.

2. So arbeitet der Gate

Zwei Betriebsarten

Blockierend, Vorabtest ist die Voreinstellung. Frage und Urteil erscheinen, bevor Ninja startet.

- `j`, `ja`, `y` oder `yes` lässt den Build zu, auch wenn das Modell `likely_fail` sagt. Dieses Ja wird für genau diesen Stand gemerkt.
- Die Eingabetaste, `n` oder jede andere Eingabe stoppt nur diesen Lauf. Sie wird nicht gemerkt, der nächste `m` fragt wieder.
- Sieht das Modell einen Compile-Bruch, ohne selbst eine Frage zu stellen, lautet die Frage `Sieht nicht kompilierbar aus. Trotzdem bauen?`.
- Ein Stand, der schon einmal fehlgeschlagen ist, startet trotzdem. Der Gate sagt das und fragt das Modell dafür nicht noch einmal.
- Läuft `m` ohne Terminal, etwa aus einer IDE-Aufgabe ohne Eingabe, wird die Frage nur gedruckt, und der Build startet.

Parallel, nur Hinweis startet den Build sofort, zusammen mit der Anfrage, und bricht ihn von selbst nie ab.

- Hinweise erscheinen im Terminal und können im Ninja-Log nach oben rutschen. Nach dem Build stehen sie noch einmal gesammelt zwischen `----- ai-patch-gate -----`.
- Wird der Build fertig, während die Anfrage noch läuft, bricht der Gate die Anfrage ab. Der nächste `m` prüft erneut. Es gibt keine Wartezeit und keine Sperre.
- Unter Windows öffnet ein konkreter Verdacht zusätzlich das Hinweisfenster aus Abschnitt 3. Der Build läuft dabei weiter.
- `Strg-C` im Terminal und der Stop-Knopf von VS Code beenden den Build.

Die einzige harte Weigerung ist `AAOS_AI_GATE=stop`. Sie gilt nur in der blockierenden Betriebsart und nur bei `likely_fail`. Dann startet der Build nicht, und es wird nicht gefragt. Die vom Installer geschriebene Konfiguration setzt `AAOS_AI_GATE=on`, also nie die harte Weigerung.

Was der Gate sich merkt

Was	Wo	Wann es verschwindet
Ihr Ja auf eine Rückfrage, gebunden an den genauen Stand	<code>out/.aaos-ai-gate/answers</code> in der Distribution	<code>m clean</code> löscht nur dieses Verzeichnis.
Welche Dateien das Modell schon gesehen hat und ob ein Stand kompiliert hat	<code>~/.cache/aaos-ai-gate</code> in der Distribution	Bleibt erhalten, auch bei <code>m clean</code> und nach dem Entfernen des Gates.

Ein Abbruch mit Strg-C oder ein Rückgabewert ab 128 wird nicht als Compile-Fehler gemerkt. Eine abgebrochene Modellanfrage ebenfalls nicht: der nächste `m` prüft erneut.

Was an den Anbieter geht

Der Gate schickt den Diff mit 20 Zeilen Kontext, den obersten Stash in gekürzter Form und die Betreffzeilen der letzten Commits desselben Autors. Daraus leitet das Modell die Absicht ab.

- Claude mit API-Token spricht `api.anthropic.com` an, Antigravity mit Token die Google-Schnittstelle.
- Microsoft Copilot und jedes Abonnement laufen über die jeweilige Kommandozeile in der Distribution und deren bestehende Anmeldung.
- Eine eigene URL ersetzt den Anbieter. Dann geht alles an diese URL, und ein Token ist Pflicht.

Die eigenen Dateien des Gates schickt er nicht mit: das Skript, das Dialogskript, die Konfiguration und ein `m`, das nur den Hook enthält. Eine echte Änderung an `m` geht dagegen mit. Fehlt `jq` in der Distribution, prüft der Gate nichts, sagt das, und der Build startet trotzdem.

3. Das Hinweisfenster im parallelen Modus

Das Fenster erscheint nur unter Windows und nur in der parallelen Betriebsart, wenn das Modell etwas Konkretes meldet: eine Rückfrage, einen wahrscheinlichen Compile-Bruch, eine absehbare Laufzeitfolge oder einen verfehlten Effekt. Ein unauffälliger Diff öffnet nichts. Der Build läuft in jedem Fall weiter. Minimieren legt das Fenster in die Taskleiste, Escape und Schließen tun dasselbe wie das X.

Die folgende Skizze zeigt das Fenster schematisch. Die Nummern finden Sie in der Legende darunter wieder. Jedes Element hat einen Tooltip, der etwa zwanzig Sekunden stehen bleibt.

Die Skizze zeigt ein Dialogfenster mit dem Titel 'AAOS AI-Patch-Gate' und einem Schließen-Knopf (X) in der oberen rechten Ecke. Der Inhalt ist wie folgt strukturiert:

- 1** (Kopfzeile): Ein Textfeld mit der Meldung: 'Der Build läuft weiter. Das Modell meldet einen Verdacht, es bricht den Build nicht ab.'
- 2** (Befund): Ein Bereich mit der Überschrift 'Was das Modell erwartet', gefolgt von einer Rückfrage, Begründung und drei Einschätzungen (Compile, Laufzeit, Effekt). Ein Text am Ende besagt: 'Das Modell hat dafür keinen Beweis. Der Compiler prüft weiter.'
- 3** (Verlauf): Ein Bereich mit der Überschrift 'Rückfrage an die KI', gefolgt von einer KI-Antwort und einer Folge von Fragen und Antworten. Ein gelber Button 'Schickt die Frage ab. Der Build wartet nicht.' befindet sich rechts neben dem Text.
- 4** (Eingabezeile): Ein Textfeld mit der Eingabeaufforderung 'Warum ist der Diff riskant?'.
- 5** (Fragen): Ein blauer Button mit der Aufschrift 'Fragen'.
- 6** (Nicht mehr anzeigen): Ein Kontrollkästchen mit der Beschriftung 'Nicht mehr anzeigen'.
- 7** (Auf blockierend umstellen): Ein blauer Button mit der Aufschrift 'Auf blockierend umstellen'.
- 8** (Schließen): Ein blauer Button mit der Aufschrift 'Schließen'.
- 9** (Statuszeile): Ein Textfeld am unteren Rand mit der Meldung: 'Ab dem nächsten m blockierend. Dieser Build läuft weiter.'

Skizze des Hinweisfensters, keine Bildschirmaufnahme. Nummern wie in der Legende.

- 1 Kopfzeile.** Sagt, dass der Build weiterläuft und das Modell nur einen Verdacht meldet. Das Fenster wartet nicht auf den Compiler und hält ihn nicht an.
- 2 Befund, nur lesen.** Oben die Rückfrage, dann die Begründung sowie die drei Einschätzungen Compile, Laufzeit und Effekt. Das Modell hat dafür keinen Beweis, es kann sich irren.
- 3 Verlauf.** Beginnt mit demselben Befund. Darunter stehen Ihre Fragen und die Antworten. Es antwortet derselbe Anbieter mit derselben Anmeldung wie bei der Prüfung. Der Verlauf bleibt in diesem Fenster.
- 4 Eingabezeile.** Eine Frage, zum Beispiel warum das Modell den Diff für riskant hält. Die Eingabetaste schickt sie ab, genau wie der Knopf daneben.
- 5 Fragen.** Schickt die Zeile an denselben Anbieter, der Build wartet nicht. Ist kein Anbieter hinterlegt, ist der Knopf abgeschaltet, und der Tooltip sagt das. Während die Antwort unterwegs ist, steht das in der Statuszeile. Scheitert eine Rückfrage, zeigt das Fenster die Fehlermeldung an, nimmt sie aber nicht in den Verlauf auf.
- 6 Nicht mehr anzeigen.** Wirkt beim Schließen. Legt die Datei `.aaos-ai-gate.dialog-off` in die Wurzel des Quellbaums. Existiert eine Konfiguration im Baum, setzt es dort zusätzlich `AAOS_AI_GATE_DIALOG=off`. Entfernen Sie beides, kommt das Fenster beim nächsten Verdacht wieder.
- 7 Auf blockierend umstellen.** Schreibt `AAOS_AI_GATE_MODE=blocking` in die Konfiguration im Baum. Ab dem nächsten m fragt der Gate vor dem Build. Der laufende Build läuft zu Ende. Steht `AAOS_AI_GATE_MODE` in Ihrer Shell, gewinnt die Shell, und die Statuszeile sagt das.

- 8 **Schließen.** Auch Escape und das X. Der Build läuft weiter. Der nächste Verdacht öffnet das Fenster wieder, solange Nicht mehr anzeigen aus ist. Beim Schließen wird das temporäre Verzeichnis dieses Fensters gelöscht.
- 9 **Statuszeile.** Zeigt die letzte Aktion: Frage unterwegs, Modus gewechselt, die Shell-Variable gewinnt, oder eine Anfrage ist fehlgeschlagen. Der Build läuft in jedem dieser Fälle weiter.

4. Der Installer

Starten

In PowerShell, im Verzeichnis aaos-ai-gate:

```
powershell -NoProfile -STA -ExecutionPolicy Bypass -File .\install-windows.ps1
```

Der Installer ist eine Windows-Oberfläche. Er schreibt ausschließlich in den angegebenen Quellbaum. Ihre Windows-Benutzerumgebung und die `~/ .bashrc` der Distribution lässt er unangetastet. Solange Sie nicht auf Installieren klicken, ändert sich am Quellbaum nichts. Das X schließt das Fenster.

Vor dem Kopieren prüft der Installer, ob `jq` in der Distribution vorhanden ist. Auf dem Token- oder URL-Weg prüft er auch `curl`. Fehlt eines davon oder ist WSL nicht erreichbar, bricht er ab, ohne den Baum zu ändern, und nennt den passenden Befehl, zum Beispiel `sudo apt install jq`.

Die Felder im Fenster

Die Skizze zeigt den Installer schematisch. Die Nummern finden Sie in der Legende darunter wieder.

AAOS AI-Patch-Gate X

Quellbaum (Verzeichnis mit build/envsetup.sh)

Durchsuchen 1

Betriebsmodus

2

Anbieter

3

In Ubuntu gefunden: Claude.

Anmeldung

4

API-Token

5

☐ Abweichende Verbindung angeben 6

☐ ccache in `~/ .bashrc` (nur zusammen mit Umgebung selbst eintragen) 7

CCACHE_DIR (Linux-Pfad) Groesse

☐ Umgebung selbst eintragen, statt einer Konfiguration im Baum 8

Es wird nur der Quellbaum beschrieben. `~/ .bashrc` bleibt unangetastet. `ccache` nur dort, nicht in der Gate-Datei. 9

Installieren Entfernen 10

11

Installiert in `\wsl$\Ubuntu\home\name\aosp`.
Modus: blockierend, Vorabtest.
Anbieter: Claude, Anmeldung: Abonnement.
Konfiguration: `\wsl$\Ubuntu\home\name\aosp\ .aaos-ai-gate.conf`
m ruft den Gate nur auf, wenn diese Datei oder `AAOS_AI_GATE` gesetzt ist.

Skizze des Installers, keine Bildschirmaufnahme. Nummern wie in der Legende.

- 1 Quellbaum.** Am besten ein `\wsl$\<Distribution>\...-Pfad`, zum Beispiel `\wsl$\Ubuntu\home\<name>\aosp`. Durchsuchen öffnet die Windows-Ordnerauswahl. Installieren verlangt `build/soong/bin/m`, `build/soong/soong_ui.bash` und `build/envsetup.sh`.
- 2 Betriebsmodus.** Vorausgewählt ist blockierend, Vorabtest. Der zweite Eintrag ist parallel, nur Hinweis. Die Wahl wird als `AAOS_AI_GATE_MODE` in die Konfiguration geschrieben.

- 3 **Anbieter.** Claude, Microsoft Copilot oder Antigravity. Beim Öffnen und nachdem der Quellbaum feststeht, sucht der Installer in der Distribution nach `claude`, `copilot` und `agy`. Gefundene Einträge heißen „installiert“, die anderen „nicht gefunden“, darunter steht die Kurzliste. Vorausgewählt ist die erste gefundene Kommandozeile in der Reihenfolge Claude, Microsoft Copilot, Antigravity. Liegt der Baum unter `\\ws1$\`, gilt diese Distribution, sonst die Standard-Distribution. Ist keine Kommandozeile da, bleibt Claude vorausgewählt. Fehlt die Kommandozeile zum gewählten Abonnement, steht das im Ergebnisfeld, die Installation läuft trotzdem.
- 4 **Anmeldung.** Vorausgewählt ist Abonnement, vorhandene Anmeldung: die Kommandozeile des Anbieters ist in der Distribution bereits angemeldet. Der zweite Eintrag ist API-Token.
- 5 **API-Token.** Bei Abonnement ist das Feld grau und wird nicht gebraucht. Bei API-Token sind die Zeichen verdeckt. Ein leeres Token oder eines mit Zeilenumbruch wird abgelehnt. Das Token landet nur in der Konfiguration im Quellbaum.
- 6 **Abweichende Verbindung angeben.** Ausgeschaltet bleiben Modellname und Chat-Completions-URL unsichtbar, denn sie sind keine Voraussetzung. Eingeschaltet erscheinen beide Felder. Leer lassen heißt: Voreinstellung des Anbieters. Eine eingetragene URL ersetzt den Anbieter und braucht ein API-Token, weil die Anmeldung der Kommandozeile dort nicht gilt.
- 7 **ccache in ~/.bashrc.** Nur wählbar, wenn Punkt 8 eingeschaltet ist, sonst bleiben Kasten und Felder grau. Einzelheiten in Abschnitt 4.4.
- 8 **Umgebung selbst eintragen.** Eingeschaltet entsteht keine Konfigurationsdatei im Baum, eine vorhandene wird entfernt. Stattdessen zeigt das Ergebnisfeld die Zeilen, die Sie selbst in `~/.bashrc` der Distribution eintragen, und als zweite Variante die PowerShell-Befehle samt `WSLENV`. Einzelheiten in Abschnitt 4.3.
- 9 **Hinweis, immer sichtbar.** Beschrieben wird nur der Quellbaum. `~/.bashrc` und die Windows-Variablen bleiben unangetastet, außer Sie führen die Befehle aus dem Ergebnisfeld selbst aus. `ccache` gehört in diese Zeilen, nicht in die Gate-Datei.
- 10 **Installieren und Entfernen.** Installieren, der gefüllte Knopf, kopiert die Skripte, trägt den Aufruf in `build/soong/bin/m` ein und schreibt die Konfiguration, sofern Punkt 8 aus ist. Der Umriss-Knopf Entfernen daneben baut den Gate wieder aus, siehe Abschnitt 4.5.
- 11 **Ergebnisfeld, nur lesen.** Nach einem normalen Lauf stehen hier Pfad, Betriebsart, Anbieter und der Ort der Konfiguration. Ein selbst gewähltes Modell erscheint nur, wenn es eingetragen wurde. Fehlermeldungen, Warnungen und die Anleitung zum Selbsteintragen erscheinen ebenfalls hier.

4.1 Was der Installer in den Baum schreibt

- `build/soong/bin/ai-patch-gate.sh`, das Gate-Skript, und `build/soong/bin/aaos-ai-gate-dialog.ps1`, das Hinweisfenster. Das Gate-Skript wird dabei mit Unix-Zeileneenden geschrieben, auch wenn es aus einem Windows-Checkout mit CRLF stammt.
- Einen kurzen Block in `build/soong/bin/m`, direkt vor dem bestehenden Build-Aufruf. Der Block ruft den Gate über `_wrap_build` auf, damit die gewohnte Zeile „#### build completed successfully ...“ samt Bauzeit erhalten bleibt. Lehnt der Gate einen Build ab, erscheint stattdessen „failed to build some targets“.
- Die Konfiguration `.aaos-ai-gate.conf` in der Wurzel des Baums, sofern Sie die Umgebung nicht selbst eintragen. Sie enthält nur `AAOS_AI_GATE` und Namen, die mit `AAOS_AI_GATE_` beginnen. Andere Zeilen, etwa `PATH`, `LD_PRELOAD` oder `ccache`-Variablen, ignoriert der Gate. Variablen aus Ihrer Shell gewinnen gegenüber der Datei.

Fehlt in `m` die erwartete Zeile `_wrap_build "$TOP/build/soong/soong_ui.bash"`, bleiben die Skripte im Baum, und das Ergebnisfeld zeigt den Block, den Sie von Hand einsetzen. Findet der Installer einen älteren Block aus einer früheren Version, bringt er ihn auf die aktuelle Form.

Damit `repo` status den Hook nicht als lokale Änderung meldet, trägt der Installer die beiden Skripte in die Git-Ausschlussliste von `build/soong` ein und markiert `bin/m` mit `skip-worktree`. Rückgängig in WSL:

```
git -C build/soong update-index --no-skip-worktree bin/m
```

`m` ruft den Gate nur auf, wenn die Konfiguration existiert oder `AAOS_AI_GATE` beziehungsweise `AAOS_AI_GATE_MODE` in der Umgebung gesetzt ist. Ohne beides baut `m` wie vorher.

4.2 Dateirechte

Liegt der Baum unter `\\wsl$`, setzt der Installer über WSL die Unix-Rechte: 755 für das Skript, 600 für die Konfiguration, damit nur Ihr Benutzer das Token lesen kann. Liegt der Baum auf einem Windows-Laufwerk, das in WSL unter `/mnt/<laufwerk>` erscheint, gilt `chmod` dort nicht. Der Installer beschränkt die Konfiguration dann über `icaccls` auf Ihren Windows-Benutzer und weist im Ergebnisfeld darauf hin, dass der Unix-Modus 600 auf diesem Laufwerk nicht gilt.

4.3 Umgebung selbst eintragen

Wer keine Datei mit Token im Quellbaum möchte, schaltet Punkt 8 ein. Der Installer kopiert dann nur die Skripte und den Hook und zeigt im Ergebnisfeld zwei gleichwertige Varianten:

- Zeilen für `~/.bashrc` in der Distribution, zum Beispiel `export AAOS_AI_GATE=on`, `export AAOS_AI_GATE_MODE=blocking`, `export AAOS_AI_GATE_PROVIDER=claude`, `export AAOS_AI_GATE_AUTH=subscription`.
- PowerShell-Befehle, die dieselben Namen als Windows-Benutzervariablen setzen und über `WSLENV` nach WSL durchreichen. Einen schon vorhandenen `WSLENV` hängen Sie an, statt ihn zu ersetzen; die gezeigte Zeile `if ($existing) { $gate = "${existing}:$gate" }` tut genau das. Das Suffix `/u` reicht den Wert nach WSL durch, ohne ihn als Pfad zu übersetzen.

Diese Befehle führen Sie selbst aus. Ohne die Variablen baut `m` wie vorher.

4.4 ccache

`ccache` beschleunigt wiederholte C- und C++-Übersetzungen. AOSP liefert kein `ccache` mehr mit und erwartet drei Variablen in der Umgebung: `USE_CCACHE=1`, `CCACHE_EXEC` mit dem Pfad des Programms und `CCACHE_DIR` mit dem Cache-Verzeichnis. Alles Weitere, `compilercheck` und `Sloppiness`, setzt `build/make/core/ccache.mk` selbst.

Der Installer bietet den `ccache`-Kasten nur zusammen mit „Umgebung selbst eintragen“ an, und die Zeilen gehören ausdrücklich in `~/.bashrc`, nicht in die Gate-Datei. Der Grund: Die Gate-Datei liest nur der Hook in `build/soong/bin/m`. `mm`, `mmm`, `mma` und `mma` sind eigene Skripte, und `make` aus `envsetup.sh` ruft `soong_ui.bash` direkt. Stünde `USE_CCACHE` nur in der Gate-Datei, sähen diese Wege ein anderes `CC_WRAPPER`, und bei jedem Wechsel erzeugten Soong und Kati den Build neu. Nach dem Entfernen des Gates wäre `ccache` zudem still verschwunden, und der nächste Build lief kalt.

Verzeichnis und Größe sind Linux-Pfade, Voreinstellung `~/.cache/ccache` und 100G. `~`, `~/`, `$HOME` und `${HOME}` werden in der Distribution zum Home-Verzeichnis. `CCACHE_EXEC` ist der Pfad, den `command -v ccache` dort liefert. Liegt der Baum unter `\\wsl$`, führt der Installer `ccache -M <Größe>` in dieser Distribution aus; `ccache` merkt sich die Größe selbst. Liegt der Baum auf einem Windows-Laufwerk, steht der Befehl zum Selbstaussführen im Ergebnisfeld. Fehlt das Paket, setzt der Installer `USE_CCACHE` nicht und nennt `sudo apt-get install -y ccache`. `CCACHE_MAXSIZE` schreibt er nicht.

4.5 Entfernen

Entfernen nimmt den Hook aus `m`, löscht die beiden Skripte, die Konfiguration und die Datei `.aaos-ai-gate.dialog-off`, bereinigt die Ausschlusslisten und hebt `skip-worktree` auf. Der Diff-Cache in `~/.cache/aaos-ai-gate` und das Verzeichnis `out/` bleiben unangetastet. Lässt sich der Hook in `m` nicht sauber entfernen, etwa weil er von Hand verändert wurde, bleiben Skripte und Konfiguration stehen, und das Ergebnisfeld bittet Sie, `m` von Hand zu prüfen. So entsteht nie ein `m`, das ein gelöscht Skript aufruft.

Im Fenster ist Entfernen der Umriss-Knopf neben Installieren. Ohne Fenster, mit gesetztem Quellbaum:

```
$env:AAOS_INSTALL_TREE = '\\wsl$\Ubuntu\home\<name>\aosp'  
powershell -NoProfile -STA -ExecutionPolicy Bypass -File .\install-windows.ps1 -Uninstall
```

5. Installation von Hand

Wer den Installer nicht verwenden möchte, spielt den Patch `aaos-ai-patch-gate.diff` ein. Er enthält das Gate-Skript, das Dialogskript und den Hook in `m`, nicht aber die Installer. In WSL, von der Wurzel des Checkouts:

```
patch -p1 < /mnt/c/Users/<name>/devel/deliverables/aaos-ai-gate/aaos-ai-patch-gate.diff  
chmod +x build/soong/bin/ai-patch-gate.sh
```

`git apply` setzt das Executable-Bit selbst. Ohne das Dialogskript

`build/soong/bin/aaos-ai-gate-dialog.ps1` bleibt der Hinweis im parallelen Modus im Terminal.

Danach setzen Sie die Variablen in `~/.bashrc` der Distribution, hier für Claude mit Abonnement:

```
export AAOS_AI_GATE=on  
export AAOS_AI_GATE_MODE=blocking  
export AAOS_AI_GATE_PROVIDER=claude  
export AAOS_AI_GATE_AUTH=subscription
```

Mit `AAOS_AI_GATE_AUTH=token` kommt `AAOS_AI_GATE_TOKEN` dazu. Copilot heißt `copilot`, Antigravity `antigravity` oder gleichbedeutend `agy`. Dieselben Namen kann Windows über die Benutzerumgebung und `WSLENV` durchreichen. `AAOS_AI_GATE_CACHE` setzen Sie nur als Linux-Pfad in WSL. Der Hook in `m` wird aktiv, sobald `AAOS_AI_GATE` oder `AAOS_AI_GATE_MODE` gesetzt ist.

6. Referenz

Variablen

Variable	Bedeutung
AAOS_AI_GATE	on schaltet den Gate ein. stop ist die einzige harte Weigerung, nur bei likely_fail und nur in der blockierenden Betriebsart. Die Datei des Installers setzt on.
AAOS_AI_GATE_MODE	blocking für den Vorabtest. parallel, suggest und suggest-only für nur Hinweis. Jeder andere Wert gilt als blocking. Ohne Setzen gilt blocking.
AAOS_AI_GATE_DIALOG	off unterdrückt das Hinweisenfenster. Dieselbe Wirkung hat die Datei .aaos-ai-gate.dialog-off in der Wurzel des Quellbaums.
AAOS_AI_GATE_PROVIDER	claude, copilot, antigravity oder agy (gleichbedeutend mit antigravity).
AAOS_AI_GATE_AUTH	subscription für die vorhandene Anmeldung der Kommandozeile, token für ein API-Token.
AAOS_AI_GATE_TOKEN	Nur bei token oder zusammen mit einer eigenen URL.
AAOS_AI_GATE_URL	Nur auf ausdrücklichen Wunsch. Ersetzt den Anbieter, die Anmeldung der Kommandozeile gilt dort nicht, das Token ist Pflicht.
AAOS_AI_GATE_MODEL	Nur setzen, wenn Sie ein bestimmtes Modell wünschen. Sonst gilt die Voreinstellung des Anbieters, bei einer eigenen URL ohne Modellname default.
AAOS_AI_GATE_BASE	Git-Revision, gegen die der Diff gebildet wird. Voreinstellung HEAD, für einen Branch zum Beispiel origin/main.
AAOS_AI_GATE_MAX_BYTES	Obergrenze in Bytes für den neu geschickten Diff. Voreinstellung 80000.
AAOS_AI_GATE_CACHE	Anderes Verzeichnis statt ~/.cache/aaos-ai-gate, nur als Linux-Pfad. m clean löscht es nicht.
USE_CCACHE, CCache_EXEC, CCache_DIR	Nur in der Shell der Distribution, zum Beispiel ~/.bashrc. Die Gate-Datei liest sie nicht.

Meldungen im Terminal

Alle Zeilen des Gates beginnen mit ai-patch-gate:.

Meldung	Bedeutung
keeping N already compiled file(s), sending M new file(s)	Nur die M neuen Dateien gehen an das Modell.
this tree already compiled	Dieser Stand hat schon einmal kompiliert, es geht keine Anfrage hinaus.
dieser Stand ist schon einmal fehlgeschlagen	Der Build startet trotzdem, das Modell wird nicht erneut gefragt.
schon bestätigt	Ein früheres Ja gilt noch, der Build startet.
Hinweisdialog wird geöffnet, der Build läuft weiter	Das Fenster aus Abschnitt 3 erscheint.
Modell-Anfrage abgebrochen, der nächste m prüft erneut	Der Build war schneller als das Modell.
recorded build rc=...	Das Ergebnis des durchgelaufenen Builds wurde gemerkt.
build aborted	Der Abbruch wurde nicht als Ergebnis gemerkt.
jq fehlt	Es wird nichts geprüft, der Build läuft normal.

Meldung	Bedeutung
Zeitlimit (55 s) beim Aufruf von ...	Die Kommandozeile des Anbieters hat nicht innerhalb von 55 Sekunden geantwortet. Der Build läuft normal.

IDE

In VS Code ist der Gate die WSL-Shell-Aufgabe, die `m` aufruft. Der Stop-Knopf bricht den Build ab. Eine Erweiterung ist nicht nötig. Android Studio sieht den Gate, wenn `m` in seinem Terminal läuft.

```
{
  "label": "AAOS m",
  "type": "shell",
  "command": "source build/envsetup.sh && lunch sdk_car_x86_64-userdebug && m",
  "options": { "cwd": "${workspaceFolder}" },
  "problemMatcher": []
}
```

7. Wenn etwas nicht klappt

Beobachtung	Ursache und Abhilfe
Der Installer meldet, dass <code>jq</code> oder <code>curl</code> fehlt oder WSL nicht erreichbar ist.	In der Distribution <code>sudo apt install jq curl</code> ausführen und den Installer erneut starten. Der Baum wurde nicht verändert.
Das Ergebnisfeld zeigt den Hook-Block zum Einsetzen.	Ihr <code>m</code> enthält die erwartete Zeile <code>_wrap_build "\$TOP/build/soong/soong_ui.bash"</code> nicht. Setzen Sie den gezeigten Block von Hand davor.
<code>m</code> baut ohne jede Gate-Meldung.	Weder existiert <code>.aaos-ai-gate.conf</code> im Baum, noch ist <code>AAOS_AI_GATE</code> oder <code>AAOS_AI_GATE_MODE</code> in der Shell gesetzt. Prüfen Sie nach dem Selbsteintragen, ob ein neues Terminal geöffnet wurde.
Der Gate meldet, dass die Anmeldung oder der Aufruf der Kommandozeile fehlgeschlagen ist.	Melden Sie die Kommandozeile in der Distribution an, oder wechseln Sie im Installer auf API-Token. Der Build läuft in beiden Fällen weiter.
Das Hinweisenfenster erscheint nicht.	Es erscheint nur in der parallelen Betriebsart und nur bei einem konkreten Verdacht. Prüfen Sie außerdem, ob <code>.aaos-ai-gate.dialog-off</code> im Baum liegt oder <code>AAOS_AI_GATE_DIALOG=off</code> gesetzt ist.
Eine Rückfrage im Fenster schlägt fehl.	Das Fenster zeigt die Fehlermeldung des Gates. Häufig fehlt die Kommandozeile des Anbieters in der Distribution oder das Token ist ungültig. Der Build läuft weiter.
<code>repo status</code> zeigt <code>build/soong/bin/m</code> als geändert.	<code>skip-worktree</code> wurde aufgehoben oder konnte nicht gesetzt werden. Setzen Sie es in WSL mit <code>git -C build/soong update-index --skip-worktree bin/m</code> .
Nach dem Entfernen bleiben Skripte und Konfiguration im Baum.	Der Hook in <code>m</code> ließ sich nicht sauber erkennen. Entfernen Sie den Block in <code>build/soong/bin/m</code> von Hand und starten Sie Entfernen erneut.